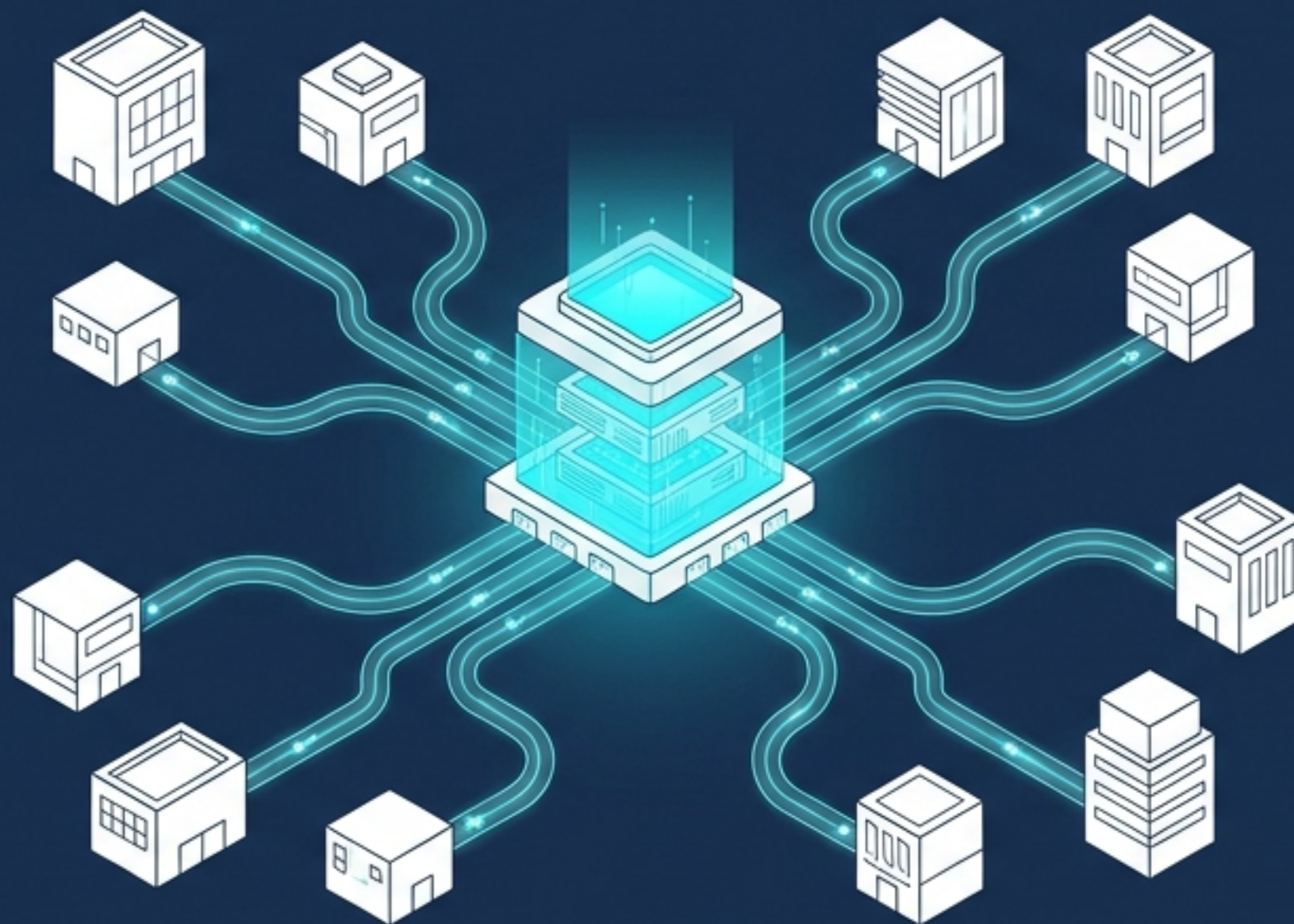


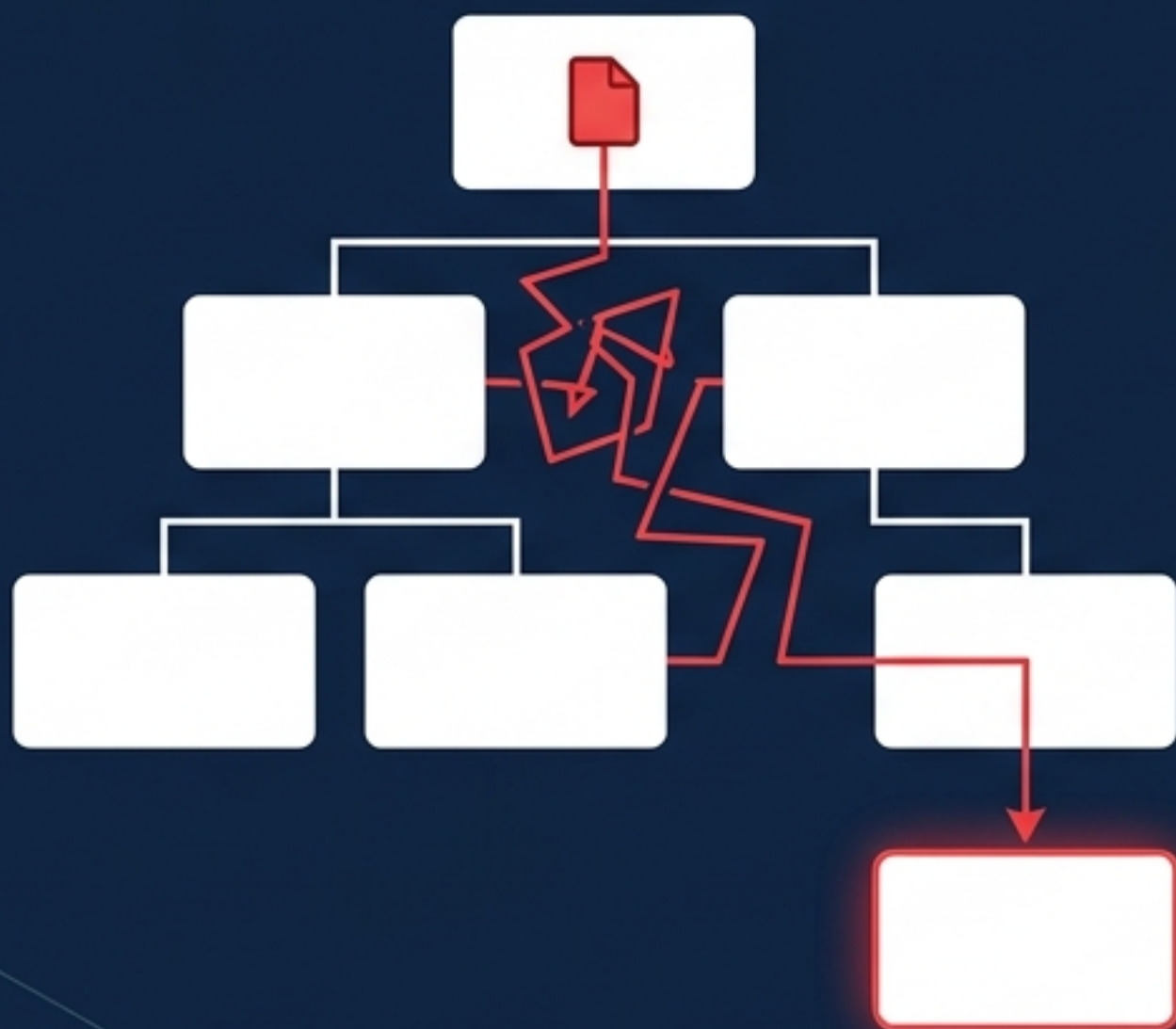
Pinia y la Arquitectura de Datos

Gestión de estado global y persistencia centralizada en Vue 3.

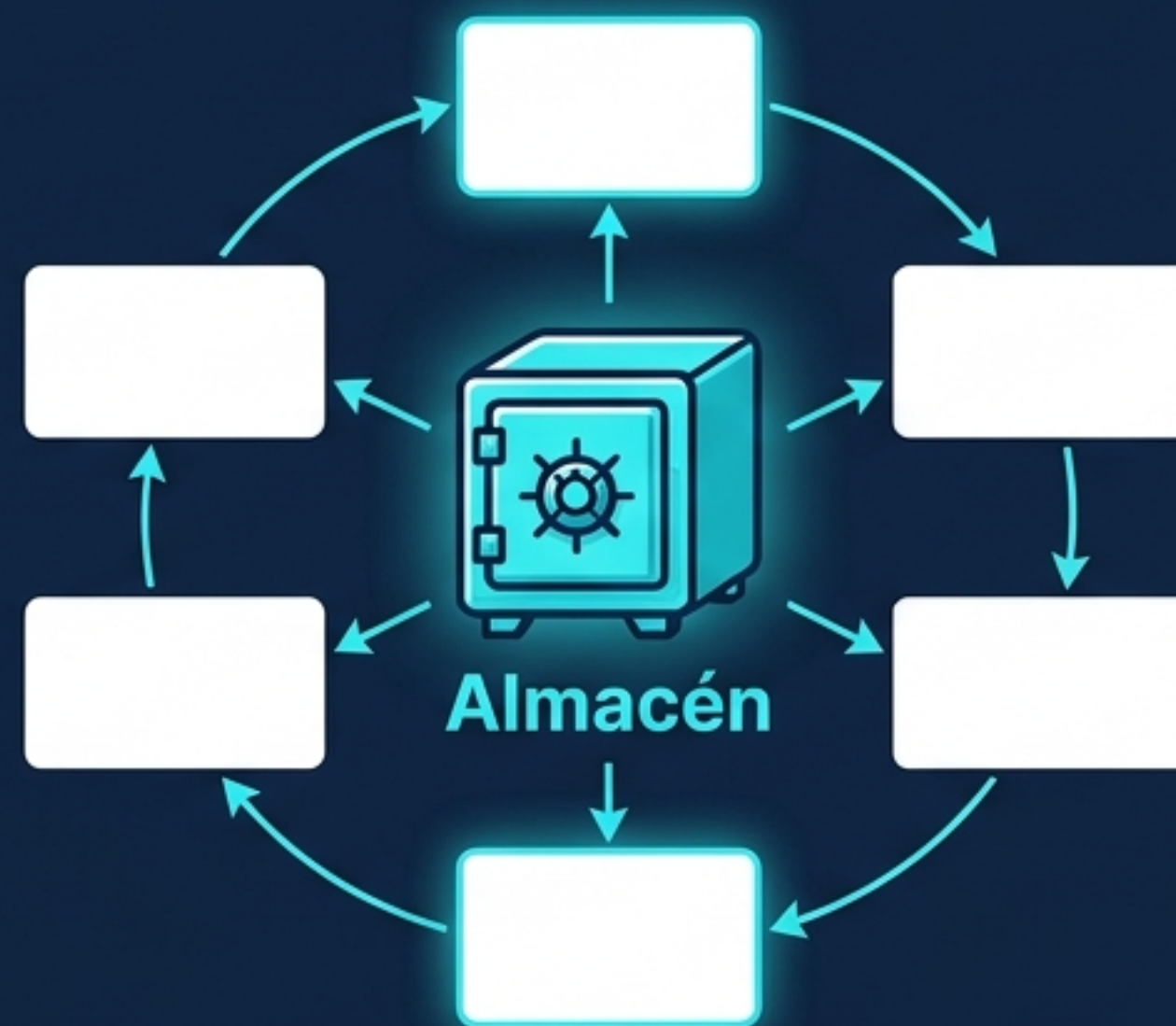


Del Caos al Orden Centralizado

El Problema: Prop Drilling



La Solución: Pinia



Un único punto de la verdad accesible desde cualquier parte de tu aplicación.

El Almacén Central (Store)



Concepto Core

Un contenedor reactivo único (Singleton) que aloja datos y lógica de negocio.

Nativo de Vue 3

Reemplaza a Vuex integrándose perfectamente con la reactividad natural del framework.

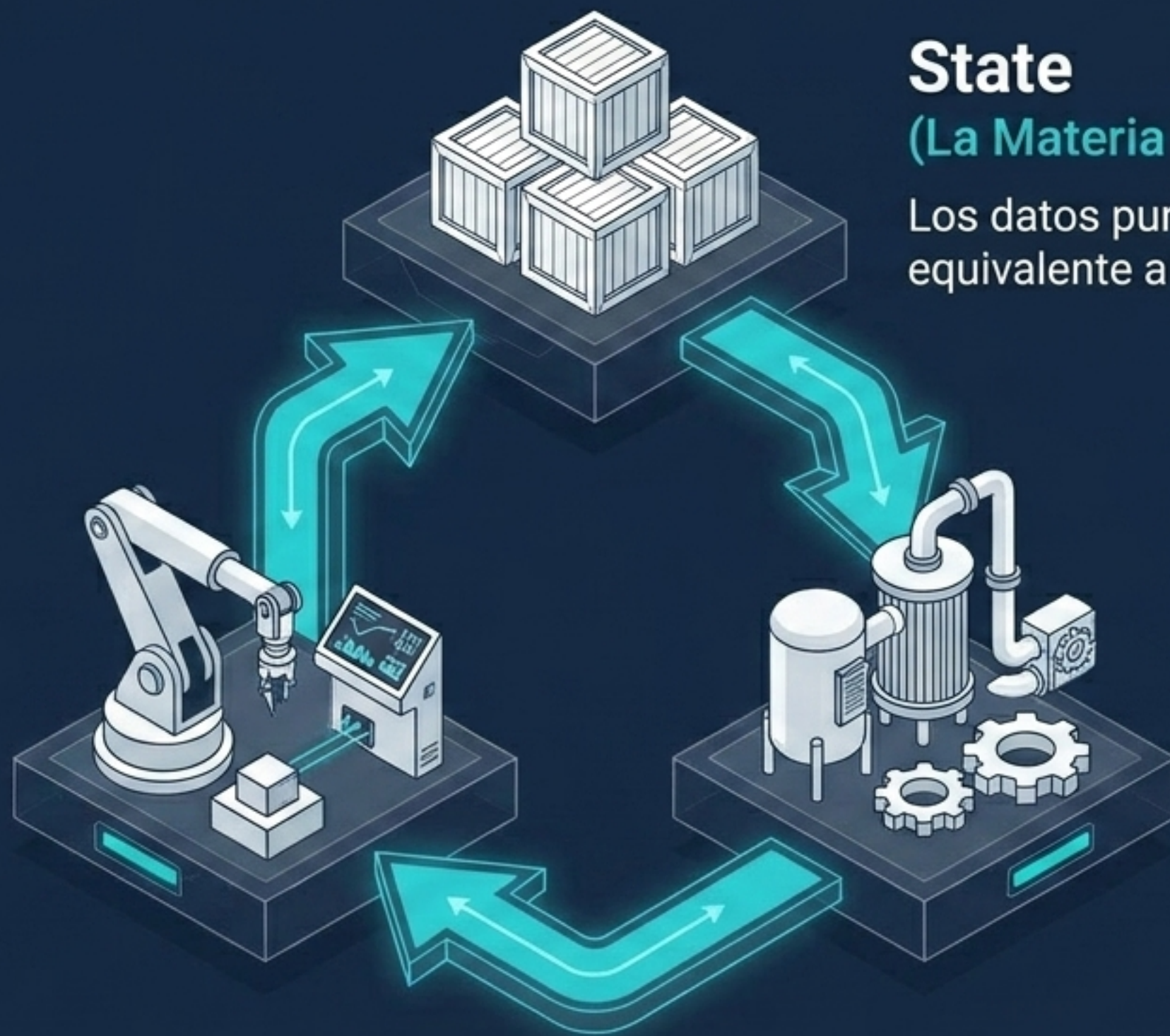
Independencia

Elimina por completo la dependencia de la jerarquía visual de componentes.

Anatomía de la Gestión de Estado

Actions
(Los Operarios)

La lógica de negocio.
Son los únicos autorizados
para modificar directamente
la materia prima.



State

(La Materia Prima)

Los datos puros y reactivos. El
equivalente a la memoria central.

Getters

(La Calculadora)

Datos derivados y
cacheados bajo demanda.
Transforman la materia
prima sin alterarla.

Sobreviviendo a la Recarga: Persistencia



El estado se restaura instantáneamente al recargar, sin consultar al servidor.



Sí Persistir (Seguro)

- Preferencias de UI (modo oscuro)
- Carritos temporales
- Filtros de búsqueda



Jamás Persistir (Peligro)

- Tokens de Autenticación (JWT)
- Contraseñas de usuario
- Información financiera

Sinergia con Inteligencia Artificial



Un Lenguaje Común

- **Contexto Explícito.** La IA entiende los contratos de datos de la store sin necesidad de leer el árbol de 50 componentes.
- **Generación Dirigida.** Es posible solicitar a un LLM el diseño de un dominio completo (ej. 'Generá la Store de Favoritos').
- **Refactorización Segura.** Al centralizar la lógica, los cambios sugeridos por la IA se propagan globalmente sin romper la interfaz gráfica.

Reglas de Oro Arquitectónicas



Una Store por Dominio

Separar responsabilidades estrictamente (Auth, Carrito, Configuración). Evitar mega-stores.



Estructuras Planas

Mantener los datos simples. Si una store necesita a otra, se interconectan sin anidar objetos infinitamente.



Mutar solo mediante Actions

Prohibido modificar la materia prima directamente desde la vista. Mantener estricta trazabilidad.



Uso Obligatorio de DevTools

Inspeccionar el estado en tiempo real para un debugging profesional y viajes en el tiempo.

Matriz de Decisión: ¿Cuándo usar qué?

	Estado Local	Composables	Pinia (Store)
Mecánica	Reactividad dentro de un solo archivo.	Funciones externas reutilizables.	Singleton centralizado.
Uso	Componentes aislados y sus hijos directos.	Lógica repetitiva sin estado global compartido.	Datos que múltiples componentes no relacionados necesitan leer o mutar.
Ejemplo Visual	Un botón de "Cargando..." o formulario.	Formateador de fechas o detector de pantalla.	Sesión activa del usuario o carrito.

“Pinia no es solo almacenamiento; es el contrato de datos definitivo de tu aplicación.”

El Desafío

Si tuviéramos que diseñar la arquitectura de una aplicación bancaria hoy... ¿Qué datos dejarían en estado local, cuáles centralizarían en Pinia, y cuáles JAMÁS persistirían en el navegador?